

UNIVERSIDAD LUTERANA SALVADOREÑA
FACULTAD DE CIENCIAS DEL HOMBRE Y
LA NATURALEZA



PARCIAL I: Desarrollo de Sitio Web con Astro Framework + tailwindcss + Enrutamiento
+ APIs

INTEGRANTES:

CARNÉ:

Katherine Alexandra De León Hernández

DH24411

Tania Lisbeth Garay Góchez

GG24326

Bertha Eunice López Martínez

LM24157

Luis Fernando Mejía Amaya

MA24353

Cristian Alexander Parada López

PL24273

CARRERA: Licenciatura En Ciencias De La Computación

MATERIA: Herramientas de Inteligencia Artificial

DOCENTE: Jorge Coto

San Salvador Centro, 1 de marzo de 2026

Índice

Introducción	3
Objetivo General	4
Objetivos Específicos	4
Contenido	5
1. Página principal.....	5
2. Sección de Ver ejercicios	6
3. Sección de El equipo.....	7
4. Sección de Pokédex	8
5. Sección Ver Detalles.....	10
6. Sección de Categorías	11
6.1 Sección de Categorías	12
7. Sección de Favoritos.....	13
Problemas Encontrados y Soluciones	15
Datos Consumidos.....	16
Ejecución en Entorno Local (Desarrollo)	16
Conclusión	17
Anexos.....	18

Introducción

La presente actividad tiene como propósito desarrollar una aplicación web completa utilizando tecnologías modernas de desarrollo frontend, integrando el framework Astro, TailwindCSS, enrutamiento dinámico y consumo de APIs externas como PokeAPI. Este proyecto está diseñado como una experiencia práctica colaborativa en la que los estudiantes pondrán en marcha habilidades fundamentales de programación interactiva, manejo del DOM, renderizado dinámico y diseño de interfaces responsivas, todo dentro de un entorno de trabajo estructurado y orientado a buenas prácticas.

A lo largo del desarrollo, el equipo construirá múltiples páginas funcionales, componentes interactivos tipo “islas”, y funcionalidades basadas en eventos y estado, culminando con la integración de datos externos y el despliegue en producción mediante Netlify. Además, el proceso incluye documentación técnica, control de versiones en plataformas como GitHub o GitLab, y la presentación audiovisual del proyecto a través de YouTube, fortaleciendo no solo competencias técnicas sino también habilidades de comunicación, trabajo en equipo y resolución de problemas.

En conjunto, esta actividad busca simular un entorno real de desarrollo profesional, permitiendo que los participantes consoliden conocimientos teóricos mediante la creación de una solución funcional, estructurada y lista para producción.

Objetivo General

Desarrollar un proyecto web integral que permita aplicar de forma práctica los fundamentos del desarrollo frontend moderno mediante el uso de herramientas, frameworks y metodologías actuales, fomentando el aprendizaje significativo, el trabajo colaborativo, la resolución de problemas y la capacidad de crear soluciones digitales funcionales, estructuradas y listas para producción.

Objetivos Específicos

Fortalecer las competencias técnicas en desarrollo web mediante la implementación de interfaces interactivas, manejo de lógica de programación, consumo de servicios externos y estructuración de aplicaciones escalables y organizadas.

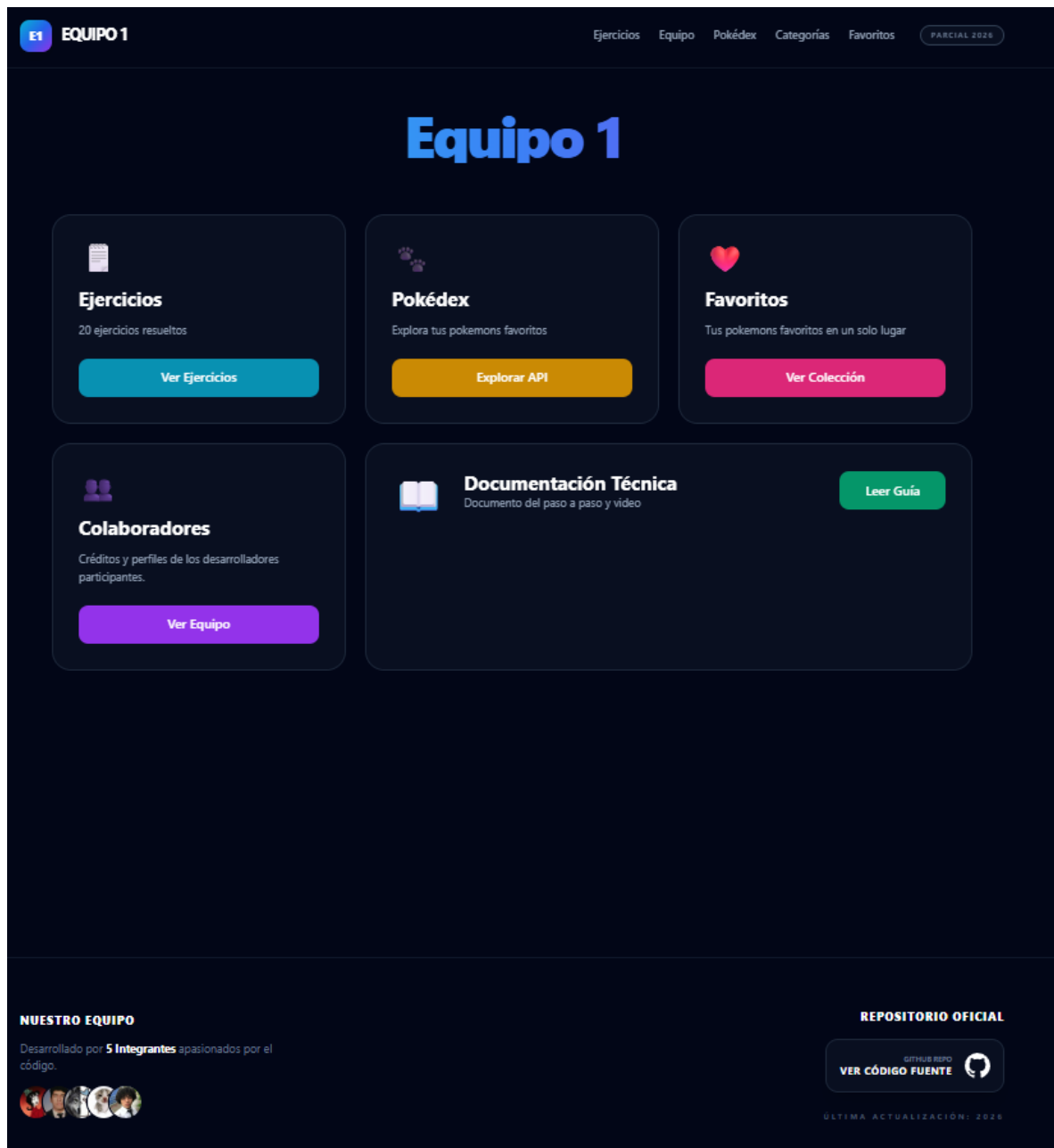
Potenciar habilidades profesionales como documentación técnica, control de versiones, presentación de proyectos, comunicación efectiva y trabajo en equipo, simulando un entorno real de desarrollo de software.

Contenido

1. Página principal

En nuestra página principal tenemos agregados los Pokémon

En nuestra página principal tenemos agregado el apartado de la página principal donde podemos ingresar a los bloques de Ejercicios, Pokédex, Favoritos, Colaboradores y Documentación Técnica.



2. Sección de Ver ejercicios

En este apartado tenemos la visualización de 20 ejercicios que podemos ejecutar.

The screenshot displays the 'Resolución #1' web application. At the top, a navigation bar includes a logo 'E1 EQUIPO 1' and links for 'Ejercicios', 'Equipo', 'Pokédex', 'Categorías', 'Favoritos', and a 'PARCIAL 2026' button. The main heading is 'Resolución #1' with the subtitle 'Explorando conceptos avanzados de JavaScript y DOM.' Below this is a 'Saludo Personalizado' section with a text input field 'Escribe tu nombre...' and a greeting 'Hola, Invitado'. A progress bar shows 'PROGRESO 1 / 20' and a 'Siguiente --' button. A grid of 20 numbered exercise buttons is shown, with button 1 highlighted. The footer contains 'NUESTRO EQUIPO' with a description and five member avatars, a 'REPOSITORIO OFICIAL' with a GitHub link and 'VER CÓDIGO FUENTE' button, and the text 'ÚLTIMA ACTUALIZACIÓN: 2026'.

3. Sección de El equipo

En este apartado podemos visualizar el equipo desarrollador.

EQUIPO 1

EjerciciosEquipoPokédexCategoríasFavoritosPARCIAL 2026

El Equipo

Desarrolladores apasionados que hicieron posible esta Pokédex interactiva utilizando Astro, Tailwind y PokéAPI.



Katy
FRONTEND DEVELOPER

"Especialista en Kotlin y Diseño. Amante de la IA y los ositos cariñosos."

[GITHUB PERFIL](#)



Luis
BACKEND DEVELOPER

"Programador Backend con experiencia en PHP y Kotlin. La IA y yo somos uno."

[GITHUB PERFIL](#)



Eunice
DISEÑADORA UI/UX

"Diseñadora UI/UX con experiencia en prototipado y desarrollo frontend."

[GITHUB PERFIL](#)



Tania
DESARROLLADORA Y MECATRÓNICA

"Desarrolladora con experiencia en Kotlin y mecatrónica. Apasionada por la robótica y la tecnología."

[GITHUB PERFIL](#)



Cristian
FULL STACK DEVELOPER

"Programador FullStack y optimización en la experiencia de usuario."

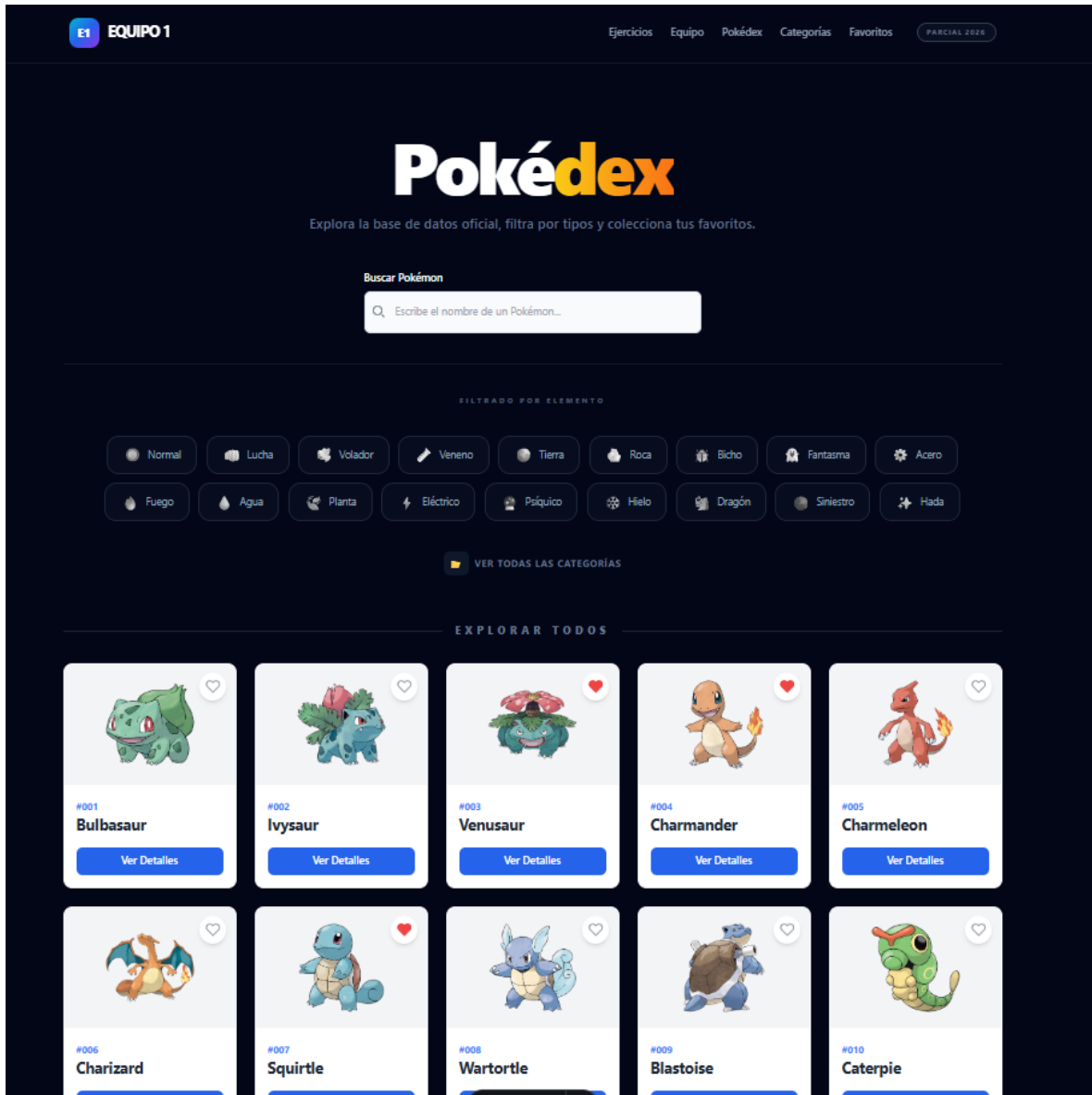
[GITHUB PERFIL](#)

PROYECTO ACADÉMICO

Hecho con ❤️ por el Equipo 1

4. Sección de Pokédex

Podemos explorar todos los Pokémon en primer plano, también podemos filtrar cada uno por elemento api la cual muestra nuestros Pokémon así mismo un buscador, como también al seleccionar **filtrado de elementos** mostrará solo los Pokémon de ese mismo filtro.



E1 EQUIPO 1

Ejercicios
Equipo
Pokédex
Categorías
Favoritos

PARCIAL 2026

#006

Charizard

Ver Detalles

#007

Squirtle

Ver Detalles

#008

Wartortle

Ver Detalles

#009

Blastoise

Ver Detalles

#010

Caterpie

Ver Detalles



#011

Metapod

Ver Detalles



#012

Butterfree

Ver Detalles



#013

Weedle

Ver Detalles



#014

Kakuna

Ver Detalles



#015

Beedrill

Ver Detalles



#016

Pidgey

Ver Detalles



#017

Pidgeotto

Ver Detalles



#018

Pidgeot

Ver Detalles



#019

Rattata

Ver Detalles



#020

Raticate

Ver Detalles

PÁGINA 1 DE 68

<

1

2

3

4

5

...

68

>

NUESTRO EQUIPO

Desarrollado por **5 Integrantes** apasionados por el código.







REPOSITORIO OFICIAL

GITHUB REPO

VER CÓDIGO FUENTE



ÚLTIMA ACTUALIZACIÓN: 2026

9

5. Sección Ver Detalles

En este apartado podremos visualizar una descripción del Pokémon así mismo las características que este contiene, también se agregó lo que es el sonido de cada evolución correspondiente al tipo de Pokémon.

E1 EQUIPO 1

[Ejercicios](#) [Equipo](#) [Pokédex](#) [Categorías](#) [Favoritos](#) [PARCIAL 2026](#)

[← Volver al listado](#)



#092

Gastly

Fantasma Veneno

"Su etéreo cuerpo está hecho de gas. Puede envolver a un oponente de cualquier tamaño hasta ahogarlo."

GRITO DE GASTLY

▶ 0:00 / 0:00



ALTURA
1.3 m

PESO
0.1 kg

EXP. BASE
62 XP

HABILIDADES
Levitación

Estadísticas Base

PS	30	
ATAQUE	35	
DEFENSA	30	
ATAQUE ESP.	100	
DEFENSA ESP.	35	
VELOCIDAD	80	

NUESTRO EQUIPO

Desarrollado por 5 Integrantes apasionados por el código.



REPOSITORIO OFICIAL

BITBUSS HUBO

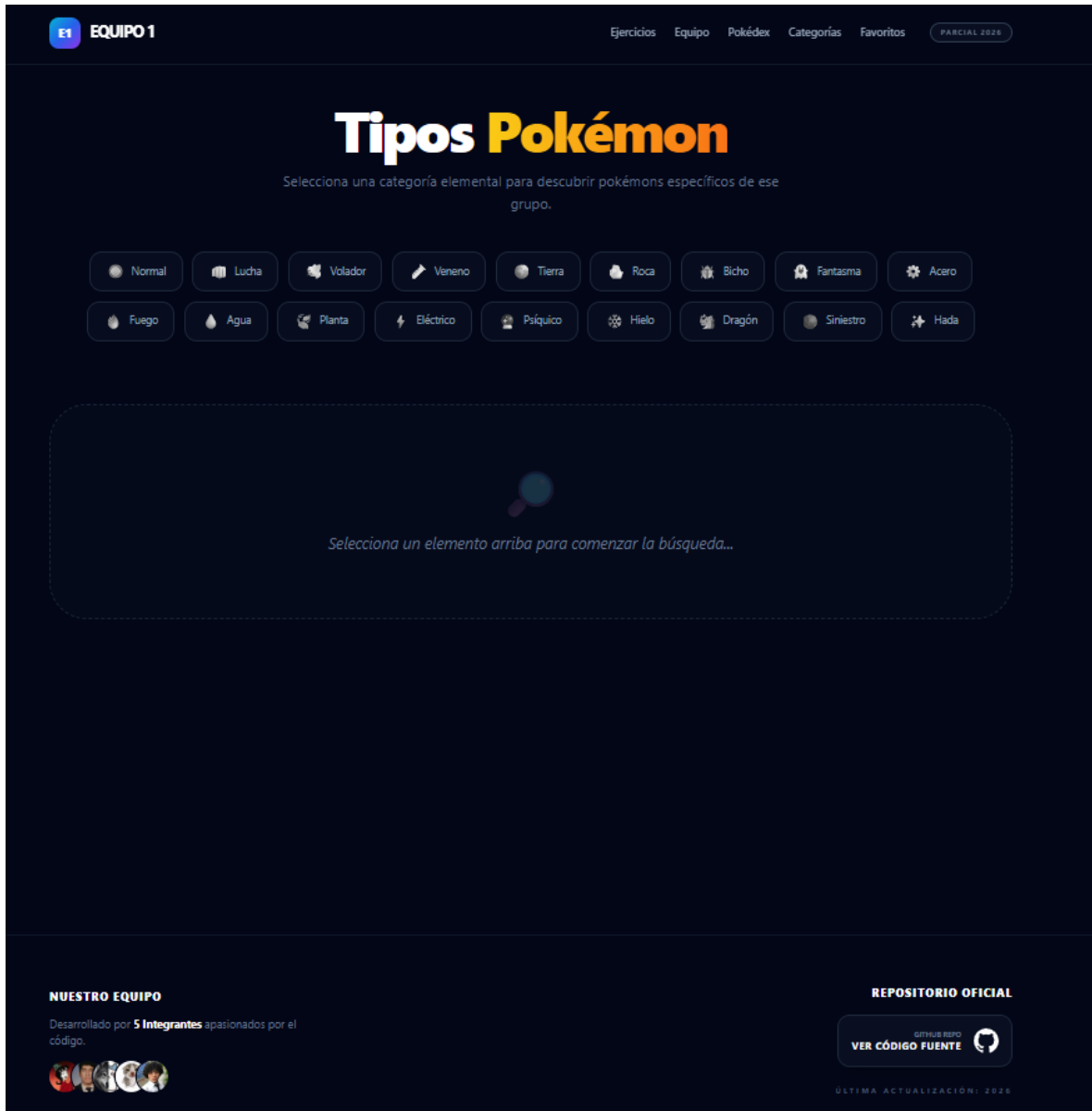
VER CÓDIGO FUENTE



ÚLTIMA ACTUALIZACIÓN: 2028

6. Sección de Categorías

En este apartado podremos ver las diferentes categorías de los tipos de Pokémon que existen actualmente.



6.1 Sección de Categorías

Al seleccionar la categoría, la página mostrará un apartado solo de las categorías, al seleccionar el tipo, puede descubrir Pokémon específico de ese grupo

E1 EQUIPO 1

EjerciciosEquipoPokédexCategoríasFavoritosPARCIAL 2024

Tipos Pokémon

Selecciona una categoría elemental para descubrir pokémons específicos de ese grupo.

Normal

Lucha

Volador

Veneno

Tierra

Roca

Bicho

Fantasma

Acero

Fuego

Agua

Planta

Eléctrico

Psíquico

Hielo

Dragón

Siniestro

Hada

Viendo Tipo: Ghost

40 resultados



#092
Gastly

Ver Detalles



#093
Haunter

Ver Detalles



#094
Gengar

Ver Detalles



#200
Misdreavus

Ver Detalles



#292
Shedinja

Ver Detalles



#302
Sableye

Ver Detalles



#353
Shuppet

Ver Detalles



#354
Banette

Ver Detalles



#355
Duskull

Ver Detalles



#356
Dusclops

Ver Detalles



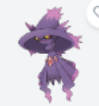
#425
Drifloon

Ver Detalles



#426
Drifblim

Ver Detalles



#476



#467



#477



#478



#479

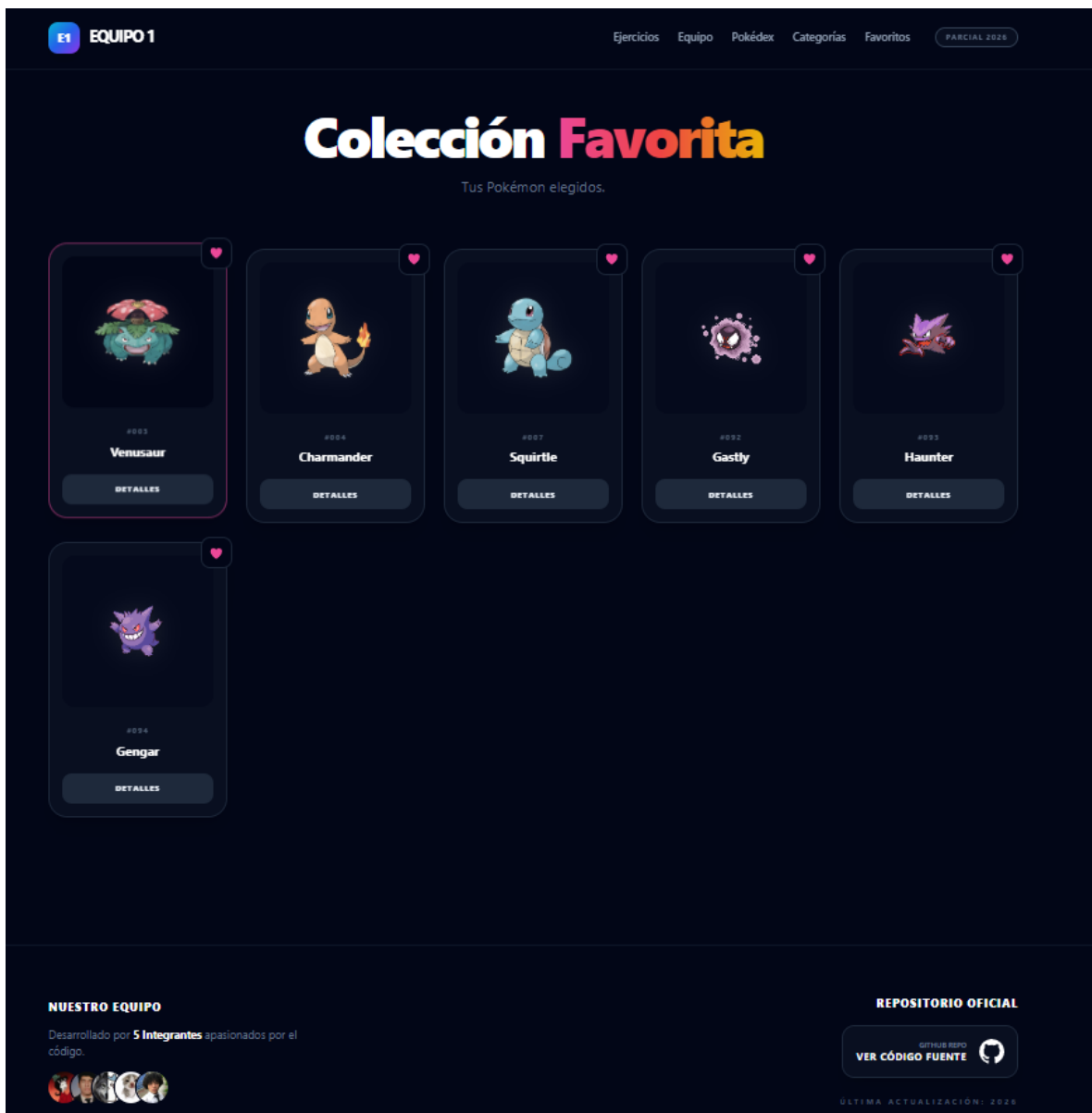


#487

12

7. Sección de Favoritos

En este apartado podremos seleccionar como “favoritos” entre todos los Pokémon.



8. Sección de Documentación Técnica

En esta sección podremos visualizar un adelanto de la documentación y también descargarla.

E1 EQUIPO 1

Ejercicios Equipo Pokédex Categorías Favoritos PARCIAL 2020

Documentación

Paso a paso del desarrollo y video explicativo.

RECURSO PRINCIPAL

Manual Técnico del Proyecto

UNIVERSIDAD LUTERANA SALVADOREÑA
FACULTAD DE CIENCIAS DEL HOMBRE Y
LA NATURALEZA



PARCIAL I: Desarrollo de Sitio Web con Astro Framework + tailwindcss + Enrutamiento + APIs

INTEGRANTES:	CARNÉ:
Katherine Alexandra De León Hernández	DH24411
Tania Lisbeth Garay Góchez	GG24326
Bertha Eunice López Martínez	LM24157
Luis Fernando Mejía Amaya	MA24353
Cristian Alexander Parada López	PL24273

CARRERA: Licenciatura En Ciencias De La Computación
MATERIA: Herramientas de Inteligencia Artificial
DOCENTE: Jorge Coto

Descargar PDF Completo

Problemas Encontrados y Soluciones

Problema: Durante la implementación del módulo de categorización, el sistema no lograba reconocer ni interpretar correctamente los tipos de Pokémon obtenidos desde la fuente de datos, lo que impedía ejecutar el proceso de filtrado dinámico basado en categorías.

Solución: Se realizó una modificación en la lógica de consulta de datos, incorporando explícitamente el atributo types dentro de la estructura de extracción en el prompt/consulta. Esta corrección permitió acceder correctamente a la propiedad que contiene los tipos de cada Pokémon, habilitando así el funcionamiento adecuado del mecanismo de filtrado por categorías.

Problema: Subir cambios por parte de cada integrante del equipo, al momento de realizar los cambios y subirlos para que los integrantes del grupo lograrán visualizarlos, estos cambios no lograban subirse y no se podía cargar cambios, dificultando avances del proyecto por cada uno de los integrantes

Solución: La solución al problema fue actualizar la versión de nuestros framework Astro, la cual se logró identificar una diferencia de versiones en cada integrante del grupo, una vez actualizado se logró visualizar cada cambio en el proyecto

Consumo de API

Para la obtención de datos dinámicos se utilizó la API pública PokeAPI, la cual permite acceder a información estructurada de cada Pokémon mediante peticiones HTTP.

La comunicación con la API se realizó utilizando la función fetch() junto con programación asíncrona (async/await) para manejar correctamente las respuestas del servidor.

Datos Consumidos

Del objeto JSON recibido desde la API, se extrajeron únicamente las siguientes propiedades:

id → Identificador numérico del Pokémon.

name → Nombre oficial del Pokémon.

Img → Imagen principal del Pokémon.

types → Tipo o tipos del Pokémon.

5.2 Procesamiento de Datos

Una vez recibida la respuesta en formato JSON:

Se accede a las propiedades necesarias del objeto.

Se almacenan temporalmente en variables.

Se renderizan dinámicamente dentro de la interfaz.

En el caso de múltiples tipos, se recorren utilizando métodos como map() para mostrarlos en pantalla.

El consumo de la API permite que la aplicación muestre información actualizada y dinámica, evitando el uso de datos estáticos.

Ejecución en Entorno Local (Desarrollo)

Actualmente, la aplicación se encuentra en fase de desarrollo y pruebas, ejecutándose en entorno local mediante el servidor de desarrollo de Astro.

Para levantar el proyecto se siguieron los siguientes pasos

- 1- Clonar el repositorio desde GitHub.
- 2- Instalar dependencias mediante el comando npm install
- 3- Ejecutar el servidor de desarrollo, con el comando npm run dev
- 4- Acceder a la aplicación desde el navegador en: <http://localhost:4321/>

Conclusión

El desarrollo de este proyecto web permitió integrar de manera práctica los conocimientos teóricos adquiridos en el área de desarrollo frontend, evidenciando la importancia de utilizar herramientas modernas como Astro, TailwindCSS y el consumo de APIs para construir aplicaciones dinámicas, funcionales y escalables. A lo largo del proceso, el equipo no solo logró implementar correctamente las diferentes secciones y funcionalidades planteadas, sino que también enfrentó desafíos técnicos reales que fortalecieron sus habilidades de análisis, resolución de problemas y adaptación a entornos de desarrollo colaborativo.

Asimismo, la experiencia demostró que el trabajo en equipo, el control de versiones y la correcta documentación son elementos fundamentales para el éxito de proyectos de software, ya que permiten mantener la organización, coherencia y eficiencia durante todo el ciclo de desarrollo. La integración de datos externos mediante PokeAPI evidenció el valor de las aplicaciones conectadas a servicios en línea, capaces de ofrecer información actualizada y mejorar la experiencia del usuario.

En síntesis, esta actividad no solo cumplió con los objetivos académicos propuestos, sino que también simuló de manera efectiva un entorno profesional real, consolidando competencias técnicas y habilidades blandas esenciales para el futuro desempeño en el campo de la informática y el desarrollo web.

Anexos

✓ PARCIALASTRO

> .astro

> .vscode

> node_modules

✓ public

| > documentacion

✓ src

> components

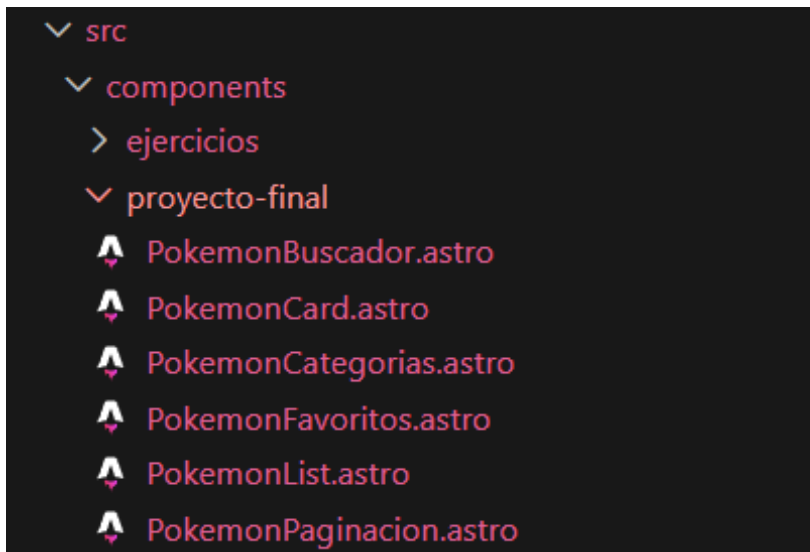
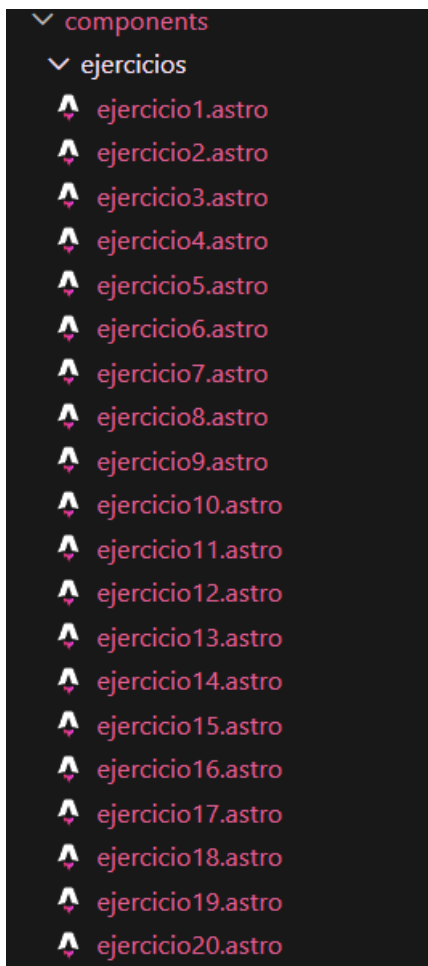
> layouts

> pages

> service

> styles

> utils



```
> layouts
  ✓ pages
    ✓ ejercicios
      🏠 [page].astro
    ✓ proyecto-final
      ✓ pokemon
        🏠 [id].astro
        🏠 categorias.astro
        🏠 favoritos.astro
        🏠 index.astro
        🏠 colaboradores.astro
        🏠 documentacion.astro
      🏠 index.astro
    ✓ service
      JS pokeApi.js
```

2. Sincronización final de la estructura

Para que todo funcione sin errores 404, tu carpeta `src/pages` debe verse así:

Plaintext

```
src/
└─ pages/
    ├── ejercicios/
    │   └─ index.astro      <-- Accesible en /ejercicios
    ├── proyecto-final/
    │   ├── index.astro    <-- Accesible en /proyecto-final
    │   └─ pokemon/
    │       ├── index.astro <-- Accesible en /proyecto-final/pokemon
    │       └─ favoritos.astro <-- Accesible en /proyecto-final/pokemon/favoritos
    └─ index.astro        <-- Tu página raíz (opcional)
```

3. Notas Importantes:

1. **Enrutamiento:** Si tu página de inicio está dentro de `/proyecto-final/index.astro`, para entrar a ella debes escribir en el navegador `localhost:4321/proyecto-final`.
2. **Layouts:** Asegúrate de que los archivos de Layout que creamos al principio estén en `src/layouts/`.
3. **Estilos:** No olvides que todos tus archivos `.astro` deben importar el CSS global (`import '.../.../styles/global.css';`) o que el Layout ya lo incluya.

Ese error de Git es un clásico. Te está diciendo que tienes cambios locales en el archivo `package-lock.json` que Git no quiere borrar al bajar lo nuevo de la rama `dev`.

Como `package-lock.json` es un archivo que se genera automáticamente, lo más sano es "limpiar" tus cambios locales para dejar que entre lo que viene del repositorio.

Cómo solucionar el error de Git

Tienes dos opciones rápidas en la terminal:

Opción A: Descartar tus cambios locales (Recomendado)

Si no has hecho cambios importantes manualmente en las dependencias, simplemente borra tus cambios locales de ese archivo y vuelve a intentar el pull:

Bash

```
git checkout package-lock.json
git pull origin dev
```

Opción B: Guardar tus cambios en un "stashed"

Si quieres conservar tus cambios por si acaso, pero quitarlos de en medio para el pull:

```
katy@Ubuntu:~/Downloads/parcial1Astro_1$ git pull origin dev
remote: Enumerating objects: 90, done.
remote: Counting objects: 100% (88/88), done.
remote: Compressing objects: 100% (44/44), done.
remote: Total 64 (delta 19), reused 60 (delta 18), pack-reused 0 (from 0)
Unpacking objects: 100% (64/64), 1.63 MiB | 4.80 MiB/s, done.
From https://github.com/CristianParadaLopez/parcial1Astro_1
* branch      dev          -> FETCH_HEAD
  8f7acd2..17b559e dev      -> origin/dev
Updating 8f7acd2..17b559e
error: Your local changes to the following files would be overwritten by merge:
    package-lock.json
Please commit your changes or stash them before you merge.
```


tanisagochez x Meet - hnb-wgjj-mam x +

https://meet.google.com/hnb-wgjj-mam

Cristian Alexander Parada López (Presentar)

Google Chrome | MManual 11CC302.docx - Docum... | CristianParadaLopez/paradaLopez... | PokéAPI - Proyecto Final

Localhost:4321/proyecto-final

PokéAPI Explorer

Busca y explora tus Pokémon favoritos

Buscar Pokémon

Escribe el nombre de un Pokémon...



#001
Bulbasaur

Ver Detalles



#002
Ivysaur

Ver Detalles



#003
Venusaur

Ver Detalles



#004
Charmander

Ver Detalles

23°C Despejado

21:51 | hnb-wgjj-mam

Cristian Alexander Parada Ló... Tania Góchez Eunice Martinez Katsup Tania Lisbeth G...

tanisagochez x Meet - hnb-wgjj-mam x +

https://meet.google.com/hnb-wgjj-mam

Eunice Martinez (Presentar)

Visual Studio Code | hnb-wgjj-mam

```

1  const authors = [
2    {
3      name: "Eunice Martinez",
4      role: "Backend Developer",
5      image: "eunice.png",
6      about: "Desarrollador frontend con experiencia en aplicaciones web y móviles. Apasionado por HTML y la inteligencia artificial.",
7      contact: {
8        phone: "7125 8614",
9        email: "eunice.martinez.123@gmail.com",
10       github: "https://github.com/eunice-martinez"
11      },
12      skills: {
13        web: ["HTML", "CSS", "JavaScript", "React", "PHP"],
14        mobile: ["Swift", "Kotlin"],
15        frameworks: ["Spring"]
16      }
17    }
18  ];
19
20  // Exportar datos a JSON
21  const jsonData = JSON.stringify(authors, null, 2);
22  console.log(jsonData);
23
24  // Crear archivo JSON
25  fs.writeFileSync('authors.json', jsonData, { encoding: 'utf-8' });
26
27  console.log('Archivo JSON creado exitosamente');
28
29  // Verificar si el archivo existe
30  fs.access('authors.json', fs.constants.F_OK, (err) => {
31    if (err) {
32      console.error('El archivo no existe');
33    } else {
34      console.log('El archivo existe');
35    }
36  });
37
38  // Leer el contenido del archivo
39  fs.readFile('authors.json', 'utf-8', (err, data) => {
40    if (err) {
41      console.error('Error al leer el archivo');
42    } else {
43      console.log('Contenido del archivo:', data);
44    }
45  });
46
47  // Guardar en GitHub
48  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
49  const newBranch = "new-branch";
50
51  // Crear una nueva rama
52  git checkout -b newBranch;
53
54  // Agregar los cambios
55  git add .;
56
57  // Commit de los cambios
58  git commit -m "Agregar autores a la API";
59
60  // Push a GitHub
61  git push origin newBranch;
62
63  // Crear un pull request
64  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
65  console.log(`Crear un pull request en GitHub: ${prUrl}`);
66
67  // Finalizar el proceso
68  console.log('Proceso completado exitosamente');
69
70  // Verificar si el archivo existe
71  fs.access('authors.json', fs.constants.F_OK, (err) => {
72    if (err) {
73      console.error('El archivo no existe');
74    } else {
75      console.log('El archivo existe');
76    }
77  });
78
79  // Leer el contenido del archivo
80  fs.readFile('authors.json', 'utf-8', (err, data) => {
81    if (err) {
82      console.error('Error al leer el archivo');
83    } else {
84      console.log('Contenido del archivo:', data);
85    }
86  });
87
88  // Guardar en GitHub
89  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
90  const newBranch = "new-branch";
91
92  // Crear una nueva rama
93  git checkout -b newBranch;
94
95  // Agregar los cambios
96  git add .;
97
98  // Commit de los cambios
99  git commit -m "Agregar autores a la API";
100
101  // Push a GitHub
102  git push origin newBranch;
103
104  // Crear un pull request
105  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
106  console.log(`Crear un pull request en GitHub: ${prUrl}`);
107
108  // Finalizar el proceso
109  console.log('Proceso completado exitosamente');
110
111  // Verificar si el archivo existe
112  fs.access('authors.json', fs.constants.F_OK, (err) => {
113    if (err) {
114      console.error('El archivo no existe');
115    } else {
116      console.log('El archivo existe');
117    }
118  });
119
120  // Leer el contenido del archivo
121  fs.readFile('authors.json', 'utf-8', (err, data) => {
122    if (err) {
123      console.error('Error al leer el archivo');
124    } else {
125      console.log('Contenido del archivo:', data);
126    }
127  });
128
129  // Guardar en GitHub
130  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
131  const newBranch = "new-branch";
132
133  // Crear una nueva rama
134  git checkout -b newBranch;
135
136  // Agregar los cambios
137  git add .;
138
139  // Commit de los cambios
140  git commit -m "Agregar autores a la API";
141
142  // Push a GitHub
143  git push origin newBranch;
144
145  // Crear un pull request
146  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
147  console.log(`Crear un pull request en GitHub: ${prUrl}`);
148
149  // Finalizar el proceso
150  console.log('Proceso completado exitosamente');
151
152  // Verificar si el archivo existe
153  fs.access('authors.json', fs.constants.F_OK, (err) => {
154    if (err) {
155      console.error('El archivo no existe');
156    } else {
157      console.log('El archivo existe');
158    }
159  });
160
161  // Leer el contenido del archivo
162  fs.readFile('authors.json', 'utf-8', (err, data) => {
163    if (err) {
164      console.error('Error al leer el archivo');
165    } else {
166      console.log('Contenido del archivo:', data);
167    }
168  });
169
170  // Guardar en GitHub
171  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
172  const newBranch = "new-branch";
173
174  // Crear una nueva rama
175  git checkout -b newBranch;
176
177  // Agregar los cambios
178  git add .;
179
180  // Commit de los cambios
181  git commit -m "Agregar autores a la API";
182
183  // Push a GitHub
184  git push origin newBranch;
185
186  // Crear un pull request
187  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
188  console.log(`Crear un pull request en GitHub: ${prUrl}`);
189
190  // Finalizar el proceso
191  console.log('Proceso completado exitosamente');
192
193  // Verificar si el archivo existe
194  fs.access('authors.json', fs.constants.F_OK, (err) => {
195    if (err) {
196      console.error('El archivo no existe');
197    } else {
198      console.log('El archivo existe');
199    }
200  });
201
202  // Leer el contenido del archivo
203  fs.readFile('authors.json', 'utf-8', (err, data) => {
204    if (err) {
205      console.error('Error al leer el archivo');
206    } else {
207      console.log('Contenido del archivo:', data);
208    }
209  });
210
211  // Guardar en GitHub
212  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
213  const newBranch = "new-branch";
214
215  // Crear una nueva rama
216  git checkout -b newBranch;
217
218  // Agregar los cambios
219  git add .;
220
221  // Commit de los cambios
222  git commit -m "Agregar autores a la API";
223
224  // Push a GitHub
225  git push origin newBranch;
226
227  // Crear un pull request
228  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
229  console.log(`Crear un pull request en GitHub: ${prUrl}`);
230
231  // Finalizar el proceso
232  console.log('Proceso completado exitosamente');
233
234  // Verificar si el archivo existe
235  fs.access('authors.json', fs.constants.F_OK, (err) => {
236    if (err) {
237      console.error('El archivo no existe');
238    } else {
239      console.log('El archivo existe');
240    }
241  });
242
243  // Leer el contenido del archivo
244  fs.readFile('authors.json', 'utf-8', (err, data) => {
245    if (err) {
246      console.error('Error al leer el archivo');
247    } else {
248      console.log('Contenido del archivo:', data);
249    }
250  });
251
252  // Guardar en GitHub
253  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
254  const newBranch = "new-branch";
255
256  // Crear una nueva rama
257  git checkout -b newBranch;
258
259  // Agregar los cambios
260  git add .;
261
262  // Commit de los cambios
263  git commit -m "Agregar autores a la API";
264
265  // Push a GitHub
266  git push origin newBranch;
267
268  // Crear un pull request
269  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
270  console.log(`Crear un pull request en GitHub: ${prUrl}`);
271
272  // Finalizar el proceso
273  console.log('Proceso completado exitosamente');
274
275  // Verificar si el archivo existe
276  fs.access('authors.json', fs.constants.F_OK, (err) => {
277    if (err) {
278      console.error('El archivo no existe');
279    } else {
280      console.log('El archivo existe');
281    }
282  });
283
284  // Leer el contenido del archivo
285  fs.readFile('authors.json', 'utf-8', (err, data) => {
286    if (err) {
287      console.error('Error al leer el archivo');
288    } else {
289      console.log('Contenido del archivo:', data);
290    }
291  });
292
293  // Guardar en GitHub
294  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
295  const newBranch = "new-branch";
296
297  // Crear una nueva rama
298  git checkout -b newBranch;
299
300  // Agregar los cambios
301  git add .;
302
303  // Commit de los cambios
304  git commit -m "Agregar autores a la API";
305
306  // Push a GitHub
307  git push origin newBranch;
308
309  // Crear un pull request
310  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
311  console.log(`Crear un pull request en GitHub: ${prUrl}`);
312
313  // Finalizar el proceso
314  console.log('Proceso completado exitosamente');
315
316  // Verificar si el archivo existe
317  fs.access('authors.json', fs.constants.F_OK, (err) => {
318    if (err) {
319      console.error('El archivo no existe');
320    } else {
321      console.log('El archivo existe');
322    }
323  });
324
325  // Leer el contenido del archivo
326  fs.readFile('authors.json', 'utf-8', (err, data) => {
327    if (err) {
328      console.error('Error al leer el archivo');
329    } else {
330      console.log('Contenido del archivo:', data);
331    }
332  });
333
334  // Guardar en GitHub
335  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
336  const newBranch = "new-branch";
337
338  // Crear una nueva rama
339  git checkout -b newBranch;
340
341  // Agregar los cambios
342  git add .;
343
344  // Commit de los cambios
345  git commit -m "Agregar autores a la API";
346
347  // Push a GitHub
348  git push origin newBranch;
349
350  // Crear un pull request
351  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
352  console.log(`Crear un pull request en GitHub: ${prUrl}`);
353
354  // Finalizar el proceso
355  console.log('Proceso completado exitosamente');
356
357  // Verificar si el archivo existe
358  fs.access('authors.json', fs.constants.F_OK, (err) => {
359    if (err) {
360      console.error('El archivo no existe');
361    } else {
362      console.log('El archivo existe');
363    }
364  });
365
366  // Leer el contenido del archivo
367  fs.readFile('authors.json', 'utf-8', (err, data) => {
368    if (err) {
369      console.error('Error al leer el archivo');
370    } else {
371      console.log('Contenido del archivo:', data);
372    }
373  });
374
375  // Guardar en GitHub
376  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
377  const newBranch = "new-branch";
378
379  // Crear una nueva rama
380  git checkout -b newBranch;
381
382  // Agregar los cambios
383  git add .;
384
385  // Commit de los cambios
386  git commit -m "Agregar autores a la API";
387
388  // Push a GitHub
389  git push origin newBranch;
390
391  // Crear un pull request
392  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
393  console.log(`Crear un pull request en GitHub: ${prUrl}`);
394
395  // Finalizar el proceso
396  console.log('Proceso completado exitosamente');
397
398  // Verificar si el archivo existe
399  fs.access('authors.json', fs.constants.F_OK, (err) => {
400    if (err) {
401      console.error('El archivo no existe');
402    } else {
403      console.log('El archivo existe');
404    }
405  });
406
407  // Leer el contenido del archivo
408  fs.readFile('authors.json', 'utf-8', (err, data) => {
409    if (err) {
410      console.error('Error al leer el archivo');
411    } else {
412      console.log('Contenido del archivo:', data);
413    }
414  });
415
416  // Guardar en GitHub
417  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
418  const newBranch = "new-branch";
419
420  // Crear una nueva rama
421  git checkout -b newBranch;
422
423  // Agregar los cambios
424  git add .;
425
426  // Commit de los cambios
427  git commit -m "Agregar autores a la API";
428
429  // Push a GitHub
430  git push origin newBranch;
431
432  // Crear un pull request
433  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
434  console.log(`Crear un pull request en GitHub: ${prUrl}`);
435
436  // Finalizar el proceso
437  console.log('Proceso completado exitosamente');
438
439  // Verificar si el archivo existe
440  fs.access('authors.json', fs.constants.F_OK, (err) => {
441    if (err) {
442      console.error('El archivo no existe');
443    } else {
444      console.log('El archivo existe');
445    }
446  });
447
448  // Leer el contenido del archivo
449  fs.readFile('authors.json', 'utf-8', (err, data) => {
450    if (err) {
451      console.error('Error al leer el archivo');
452    } else {
453      console.log('Contenido del archivo:', data);
454    }
455  });
456
457  // Guardar en GitHub
458  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
459  const newBranch = "new-branch";
460
461  // Crear una nueva rama
462  git checkout -b newBranch;
463
464  // Agregar los cambios
465  git add .;
466
467  // Commit de los cambios
468  git commit -m "Agregar autores a la API";
469
470  // Push a GitHub
471  git push origin newBranch;
472
473  // Crear un pull request
474  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
475  console.log(`Crear un pull request en GitHub: ${prUrl}`);
476
477  // Finalizar el proceso
478  console.log('Proceso completado exitosamente');
479
480  // Verificar si el archivo existe
481  fs.access('authors.json', fs.constants.F_OK, (err) => {
482    if (err) {
483      console.error('El archivo no existe');
484    } else {
485      console.log('El archivo existe');
486    }
487  });
488
489  // Leer el contenido del archivo
490  fs.readFile('authors.json', 'utf-8', (err, data) => {
491    if (err) {
492      console.error('Error al leer el archivo');
493    } else {
494      console.log('Contenido del archivo:', data);
495    }
496  });
497
498  // Guardar en GitHub
499  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
500  const newBranch = "new-branch";
501
502  // Crear una nueva rama
503  git checkout -b newBranch;
504
505  // Agregar los cambios
506  git add .;
507
508  // Commit de los cambios
509  git commit -m "Agregar autores a la API";
510
511  // Push a GitHub
512  git push origin newBranch;
513
514  // Crear un pull request
515  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
516  console.log(`Crear un pull request en GitHub: ${prUrl}`);
517
518  // Finalizar el proceso
519  console.log('Proceso completado exitosamente');
520
521  // Verificar si el archivo existe
522  fs.access('authors.json', fs.constants.F_OK, (err) => {
523    if (err) {
524      console.error('El archivo no existe');
525    } else {
526      console.log('El archivo existe');
527    }
528  });
529
530  // Leer el contenido del archivo
531  fs.readFile('authors.json', 'utf-8', (err, data) => {
532    if (err) {
533      console.error('Error al leer el archivo');
534    } else {
535      console.log('Contenido del archivo:', data);
536    }
537  });
538
539  // Guardar en GitHub
540  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
541  const newBranch = "new-branch";
542
543  // Crear una nueva rama
544  git checkout -b newBranch;
545
546  // Agregar los cambios
547  git add .;
548
549  // Commit de los cambios
550  git commit -m "Agregar autores a la API";
551
552  // Push a GitHub
553  git push origin newBranch;
554
555  // Crear un pull request
556  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
557  console.log(`Crear un pull request en GitHub: ${prUrl}`);
558
559  // Finalizar el proceso
560  console.log('Proceso completado exitosamente');
561
562  // Verificar si el archivo existe
563  fs.access('authors.json', fs.constants.F_OK, (err) => {
564    if (err) {
565      console.error('El archivo no existe');
566    } else {
567      console.log('El archivo existe');
568    }
569  });
570
571  // Leer el contenido del archivo
572  fs.readFile('authors.json', 'utf-8', (err, data) => {
573    if (err) {
574      console.error('Error al leer el archivo');
575    } else {
576      console.log('Contenido del archivo:', data);
577    }
578  });
579
580  // Guardar en GitHub
581  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
582  const newBranch = "new-branch";
583
584  // Crear una nueva rama
585  git checkout -b newBranch;
586
587  // Agregar los cambios
588  git add .;
589
590  // Commit de los cambios
591  git commit -m "Agregar autores a la API";
592
593  // Push a GitHub
594  git push origin newBranch;
595
596  // Crear un pull request
597  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
598  console.log(`Crear un pull request en GitHub: ${prUrl}`);
599
600  // Finalizar el proceso
601  console.log('Proceso completado exitosamente');
602
603  // Verificar si el archivo existe
604  fs.access('authors.json', fs.constants.F_OK, (err) => {
605    if (err) {
606      console.error('El archivo no existe');
607    } else {
608      console.log('El archivo existe');
609    }
610  });
611
612  // Leer el contenido del archivo
613  fs.readFile('authors.json', 'utf-8', (err, data) => {
614    if (err) {
615      console.error('Error al leer el archivo');
616    } else {
617      console.log('Contenido del archivo:', data);
618    }
619  });
620
621  // Guardar en GitHub
622  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
623  const newBranch = "new-branch";
624
625  // Crear una nueva rama
626  git checkout -b newBranch;
627
628  // Agregar los cambios
629  git add .;
630
631  // Commit de los cambios
632  git commit -m "Agregar autores a la API";
633
634  // Push a GitHub
635  git push origin newBranch;
636
637  // Crear un pull request
638  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
639  console.log(`Crear un pull request en GitHub: ${prUrl}`);
640
641  // Finalizar el proceso
642  console.log('Proceso completado exitosamente');
643
644  // Verificar si el archivo existe
645  fs.access('authors.json', fs.constants.F_OK, (err) => {
646    if (err) {
647      console.error('El archivo no existe');
648    } else {
649      console.log('El archivo existe');
650    }
651  });
652
653  // Leer el contenido del archivo
654  fs.readFile('authors.json', 'utf-8', (err, data) => {
655    if (err) {
656      console.error('Error al leer el archivo');
657    } else {
658      console.log('Contenido del archivo:', data);
659    }
660  });
661
662  // Guardar en GitHub
663  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
664  const newBranch = "new-branch";
665
666  // Crear una nueva rama
667  git checkout -b newBranch;
668
669  // Agregar los cambios
670  git add .;
671
672  // Commit de los cambios
673  git commit -m "Agregar autores a la API";
674
675  // Push a GitHub
676  git push origin newBranch;
677
678  // Crear un pull request
679  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
680  console.log(`Crear un pull request en GitHub: ${prUrl}`);
681
682  // Finalizar el proceso
683  console.log('Proceso completado exitosamente');
684
685  // Verificar si el archivo existe
686  fs.access('authors.json', fs.constants.F_OK, (err) => {
687    if (err) {
688      console.error('El archivo no existe');
689    } else {
690      console.log('El archivo existe');
691    }
692  });
693
694  // Leer el contenido del archivo
695  fs.readFile('authors.json', 'utf-8', (err, data) => {
696    if (err) {
697      console.error('Error al leer el archivo');
698    } else {
699      console.log('Contenido del archivo:', data);
700    }
701  });
702
703  // Guardar en GitHub
704  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
705  const newBranch = "new-branch";
706
707  // Crear una nueva rama
708  git checkout -b newBranch;
709
710  // Agregar los cambios
711  git add .;
712
713  // Commit de los cambios
714  git commit -m "Agregar autores a la API";
715
716  // Push a GitHub
717  git push origin newBranch;
718
719  // Crear un pull request
720  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
721  console.log(`Crear un pull request en GitHub: ${prUrl}`);
722
723  // Finalizar el proceso
724  console.log('Proceso completado exitosamente');
725
726  // Verificar si el archivo existe
727  fs.access('authors.json', fs.constants.F_OK, (err) => {
728    if (err) {
729      console.error('El archivo no existe');
730    } else {
731      console.log('El archivo existe');
732    }
733  });
734
735  // Leer el contenido del archivo
736  fs.readFile('authors.json', 'utf-8', (err, data) => {
737    if (err) {
738      console.error('Error al leer el archivo');
739    } else {
740      console.log('Contenido del archivo:', data);
741    }
742  });
743
744  // Guardar en GitHub
745  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
746  const newBranch = "new-branch";
747
748  // Crear una nueva rama
749  git checkout -b newBranch;
750
751  // Agregar los cambios
752  git add .;
753
754  // Commit de los cambios
755  git commit -m "Agregar autores a la API";
756
757  // Push a GitHub
758  git push origin newBranch;
759
760  // Crear un pull request
761  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
762  console.log(`Crear un pull request en GitHub: ${prUrl}`);
763
764  // Finalizar el proceso
765  console.log('Proceso completado exitosamente');
766
767  // Verificar si el archivo existe
768  fs.access('authors.json', fs.constants.F_OK, (err) => {
769    if (err) {
770      console.error('El archivo no existe');
771    } else {
772      console.log('El archivo existe');
773    }
774  });
775
776  // Leer el contenido del archivo
777  fs.readFile('authors.json', 'utf-8', (err, data) => {
778    if (err) {
779      console.error('Error al leer el archivo');
780    } else {
781      console.log('Contenido del archivo:', data);
782    }
783  });
784
785  // Guardar en GitHub
786  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
787  const newBranch = "new-branch";
788
789  // Crear una nueva rama
790  git checkout -b newBranch;
791
792  // Agregar los cambios
793  git add .;
794
795  // Commit de los cambios
796  git commit -m "Agregar autores a la API";
797
798  // Push a GitHub
799  git push origin newBranch;
800
801  // Crear un pull request
802  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
803  console.log(`Crear un pull request en GitHub: ${prUrl}`);
804
805  // Finalizar el proceso
806  console.log('Proceso completado exitosamente');
807
808  // Verificar si el archivo existe
809  fs.access('authors.json', fs.constants.F_OK, (err) => {
810    if (err) {
811      console.error('El archivo no existe');
812    } else {
813      console.log('El archivo existe');
814    }
815  });
816
817  // Leer el contenido del archivo
818  fs.readFile('authors.json', 'utf-8', (err, data) => {
819    if (err) {
820      console.error('Error al leer el archivo');
821    } else {
822      console.log('Contenido del archivo:', data);
823    }
824  });
825
826  // Guardar en GitHub
827  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
828  const newBranch = "new-branch";
829
830  // Crear una nueva rama
831  git checkout -b newBranch;
832
833  // Agregar los cambios
834  git add .;
835
836  // Commit de los cambios
837  git commit -m "Agregar autores a la API";
838
839  // Push a GitHub
840  git push origin newBranch;
841
842  // Crear un pull request
843  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
844  console.log(`Crear un pull request en GitHub: ${prUrl}`);
845
846  // Finalizar el proceso
847  console.log('Proceso completado exitosamente');
848
849  // Verificar si el archivo existe
850  fs.access('authors.json', fs.constants.F_OK, (err) => {
851    if (err) {
852      console.error('El archivo no existe');
853    } else {
854      console.log('El archivo existe');
855    }
856  });
857
858  // Leer el contenido del archivo
859  fs.readFile('authors.json', 'utf-8', (err, data) => {
860    if (err) {
861      console.error('Error al leer el archivo');
862    } else {
863      console.log('Contenido del archivo:', data);
864    }
865  });
866
867  // Guardar en GitHub
868  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
869  const newBranch = "new-branch";
870
871  // Crear una nueva rama
872  git checkout -b newBranch;
873
874  // Agregar los cambios
875  git add .;
876
877  // Commit de los cambios
878  git commit -m "Agregar autores a la API";
879
880  // Push a GitHub
881  git push origin newBranch;
882
883  // Crear un pull request
884  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
885  console.log(`Crear un pull request en GitHub: ${prUrl}`);
886
887  // Finalizar el proceso
888  console.log('Proceso completado exitosamente');
889
890  // Verificar si el archivo existe
891  fs.access('authors.json', fs.constants.F_OK, (err) => {
892    if (err) {
893      console.error('El archivo no existe');
894    } else {
895      console.log('El archivo existe');
896    }
897  });
898
899  // Leer el contenido del archivo
900  fs.readFile('authors.json', 'utf-8', (err, data) => {
901    if (err) {
902      console.error('Error al leer el archivo');
903    } else {
904      console.log('Contenido del archivo:', data);
905    }
906  });
907
908  // Guardar en GitHub
909  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
910  const newBranch = "new-branch";
911
912  // Crear una nueva rama
913  git checkout -b newBranch;
914
915  // Agregar los cambios
916  git add .;
917
918  // Commit de los cambios
919  git commit -m "Agregar autores a la API";
920
921  // Push a GitHub
922  git push origin newBranch;
923
924  // Crear un pull request
925  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
926  console.log(`Crear un pull request en GitHub: ${prUrl}`);
927
928  // Finalizar el proceso
929  console.log('Proceso completado exitosamente');
930
931  // Verificar si el archivo existe
932  fs.access('authors.json', fs.constants.F_OK, (err) => {
933    if (err) {
934      console.error('El archivo no existe');
935    } else {
936      console.log('El archivo existe');
937    }
938  });
939
940  // Leer el contenido del archivo
941  fs.readFile('authors.json', 'utf-8', (err, data) => {
942    if (err) {
943      console.error('Error al leer el archivo');
944    } else {
945      console.log('Contenido del archivo:', data);
946    }
947  });
948
949  // Guardar en GitHub
950  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
951  const newBranch = "new-branch";
952
953  // Crear una nueva rama
954  git checkout -b newBranch;
955
956  // Agregar los cambios
957  git add .;
958
959  // Commit de los cambios
960  git commit -m "Agregar autores a la API";
961
962  // Push a GitHub
963  git push origin newBranch;
964
965  // Crear un pull request
966  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
967  console.log(`Crear un pull request en GitHub: ${prUrl}`);
968
969  // Finalizar el proceso
970  console.log('Proceso completado exitosamente');
971
972  // Verificar si el archivo existe
973  fs.access('authors.json', fs.constants.F_OK, (err) => {
974    if (err) {
975      console.error('El archivo no existe');
976    } else {
977      console.log('El archivo existe');
978    }
979  });
980
981  // Leer el contenido del archivo
982  fs.readFile('authors.json', 'utf-8', (err, data) => {
983    if (err) {
984      console.error('Error al leer el archivo');
985    } else {
986      console.log('Contenido del archivo:', data);
987    }
988  });
989
990  // Guardar en GitHub
991  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
992  const newBranch = "new-branch";
993
994  // Crear una nueva rama
995  git checkout -b newBranch;
996
997  // Agregar los cambios
998  git add .;
999
1000  // Commit de los cambios
1001  git commit -m "Agregar autores a la API";
1002
1003  // Push a GitHub
1004  git push origin newBranch;
1005
1006  // Crear un pull request
1007  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
1008  console.log(`Crear un pull request en GitHub: ${prUrl}`);
1009
1010  // Finalizar el proceso
1011  console.log('Proceso completado exitosamente');
1012
1013  // Verificar si el archivo existe
1014  fs.access('authors.json', fs.constants.F_OK, (err) => {
1015    if (err) {
1016      console.error('El archivo no existe');
1017    } else {
1018      console.log('El archivo existe');
1019    }
1020  });
1021
1022  // Leer el contenido del archivo
1023  fs.readFile('authors.json', 'utf-8', (err, data) => {
1024    if (err) {
1025      console.error('Error al leer el archivo');
1026    } else {
1027      console.log('Contenido del archivo:', data);
1028    }
1029  });
1030
1031  // Guardar en GitHub
1032  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
1033  const newBranch = "new-branch";
1034
1035  // Crear una nueva rama
1036  git checkout -b newBranch;
1037
1038  // Agregar los cambios
1039  git add .;
1040
1041  // Commit de los cambios
1042  git commit -m "Agregar autores a la API";
1043
1044  // Push a GitHub
1045  git push origin newBranch;
1046
1047  // Crear un pull request
1048  const prUrl = `https://github.com/CristianParadaLopez/proyecto-final/pull/new/newBranch`;
1049  console.log(`Crear un pull request en GitHub: ${prUrl}`);
1050
1051  // Finalizar el proceso
1052  console.log('Proceso completado exitosamente');
1053
1054  // Verificar si el archivo existe
1055  fs.access('authors.json', fs.constants.F_OK, (err) => {
1056    if (err) {
1057      console.error('El archivo no existe');
1058    } else {
1059      console.log('El archivo existe');
1060    }
1061  });
1062
1063  // Leer el contenido del archivo
1064  fs.readFile('authors.json', 'utf-8', (err, data) => {
1065    if (err) {
1066      console.error('Error al leer el archivo');
1067    } else {
1068      console.log('Contenido del archivo:', data);
1069    }
1070  });
1071
1072  // Guardar en GitHub
1073  const githubRepo = "https://github.com/CristianParadaLopez/proyecto-final";
1074  const newBranch = "new-branch";
1075
1076  // Crear una nueva rama
1077  git checkout -b newBranch;
1078
1079  // Agregar los cambios
1080  git add .;
1081
1082  // Commit de los cambios
1083  git commit -m "Agregar autores a la API";
1084
1085  // Push a GitHub
1086  git push origin
```

